

Git & GitHub Installation Guide

From zero to your first push - Windows and Mac

A step-by-step guide for complete beginners

Create a free GitHub account

Install Git on Windows or Mac

Connect Git to GitHub (HTTPS or SSH)

Clone or upload your first project

No experience needed - follow along one step at a time.

Before we start: what are these two things?

Git is a free program that lives on your computer. It keeps a history of every change you make to your project files, so you can always go back, see what changed, and work with others without overwriting each other.

GitHub is a website that stores a copy of your Git projects online. It is like a cloud backup plus a social network for code: you can share your work, and other people can suggest improvements.

You need both: Git does the remembering on your computer, GitHub is the online home where your project is shared and kept safe. This guide sets up both, in this order:

- Step 1 - Create your free GitHub account (on the website).
- Step 2 - Install Git on your computer (Windows or Mac).
- Step 3 - Tell Git your name and email.
- Step 4 - Connect Git to GitHub so they can talk securely.
- Step 5 - Clone or upload your first project.

Step 1 - Create your free GitHub account

You only do this once. Have your email address handy.

1. Open your web browser and go to:

```
https://github.com
```

2. Click the Sign up button (top right corner).
3. Enter your email address, then click Continue.
4. Create a password, then click Continue. Use a strong password you do not reuse elsewhere.
5. Pick a username, then click Continue. This name is public and appears in your profile link, for example `github.com/your-username`. If your first choice is taken, GitHub will suggest similar names.
6. Solve the short puzzle (CAPTCHA) to prove you are human.
7. GitHub emails you a verification code. Open your email, find the code, and type it into the GitHub page.
8. When asked about your plan, choose the Free plan - it includes everything you need. You can skip the personalisation questions or answer them; it does not matter.

Note: Use an email address you actually check - GitHub sends the verification code there, and you will need it again if you ever reset your password.

Step 2 - Install Git on Windows

On Windows you install an app called Git for Windows. It gives you the git command plus Git Bash, a friendly terminal window where you will type the commands from this guide.

2a. Download the official installer

1. Go to the official download page:

```
https://git-scm.com/download/win
```

2. The download should start automatically (a file named something like Git-2.xx.x-64-bit.exe). If it does not, click the link 64-bit Git for Windows Setup.

2b. Run the installer

1. Double-click the downloaded .exe file and click Yes if Windows asks for permission.
2. The installer asks several questions. Clicking Next through everything is safe, but three screens are worth a second look:
 - Choosing the default editor: the default is Vim, an editor that confuses many beginners. If you have VS Code or Notepad++, pick one of those from the dropdown instead - otherwise just leave it.
 - Adjusting your PATH: leave the recommended middle option (Git from the command line and also from 3rd-party software). This lets you use git from any terminal.
 - Line endings: leave the default, Checkout Windows-style, commit Unix-style line endings. It keeps your files friendly to both Windows and Mac/Linux teammates.
3. Click Install, wait a moment, then click Finish.

2c. Alternative: install with winget

If you prefer one command (or the website is unreachable), open PowerShell or Command Prompt and run:

```
winget install --id Git.Git -e
```

This installs the exact same official Git for Windows package.

2d. Check that it worked

1. Open the Start menu, type Git Bash, and press Enter. A terminal window opens - this is where you type git commands.
2. Type the following and press Enter:

```
git --version
```

3. You should see something like git version 2.47.0.windows.1. Any version number means Git is installed correctly.

Note: If you see 'git is not recognized', close every terminal window and open a new one - terminals only notice new programs after a restart. If it still fails, re-run the installer and keep the recommended PATH option.

Step 3 - Install Git on a Mac

Three ways work well. Option A is the one most Mac developers use.

Option A (recommended): install with Homebrew

Homebrew is a popular installer tool for Mac. If you already have it, open the Terminal app and run:

```
brew install git
```

If the brew command is not found, install Homebrew first from <https://brew.sh> (one command you paste into Terminal), then run the line above.

Option B: let the Mac install it for you

1. Open Terminal (press Cmd + Space, type Terminal, Enter).
2. Type this and press Enter:

```
git --version
```

3. If Git is missing, your Mac pops up a window offering to install the Command Line Developer Tools. Click Install and wait - Git comes included with them.

Option C: the official installer

Download the .pkg installer from <https://git-scm.com/download/mac> and double-click it, following the prompts.

Check that it worked

Whichever option you used, verify in Terminal:

```
git --version
```

Any version number in the reply means you are good to go.

Step 4 - Tell Git who you are

Git stamps every change you save with your name and email. Set this once, on each computer you use. Open Git Bash (Windows) or Terminal (Mac) and run these two commands, replacing the example values with your own:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Note: Use the same email address as your GitHub account - that is how GitHub connects your commits to your profile.

Check what you saved with:

```
git config --list
```

You should see your user.name and user.email in the list.

Step 5 - Connect Git to GitHub (authentication)

When Git talks to GitHub (for example when you push your work), GitHub must know it is really you. Pick ONE of the three options below. Options A and C are both beginner-friendly - Option C is the quickest single setup because it signs you in AND configures Git in one step. Option B is the classic developer setup.

Option A (beginner-friendly): HTTPS + Git Credential Manager

Git Credential Manager (GCM) remembers your GitHub login for you. The first time you push or clone something private, a browser window pops up, you sign in to GitHub once, and you are never asked again.

- Windows: good news - GCM is installed automatically with Git for Windows. There is nothing to set up.
- Mac: install it with these two commands:

```
brew install --cask git-credential-manager
git-credential-manager configure
```

That is all. The next time you push, a GitHub sign-in window opens in your browser. Sign in once and Git remembers it.

Option B: SSH keys

An SSH key is a pair of files that work like a lock and key: the public key goes on GitHub (the lock), the private key stays secret on your computer (the key). After setup, pushes need no password at all.

1. In Git Bash / Terminal, create the key (use YOUR GitHub email):

```
ssh-keygen -t ed25519 -C "you@example.com"
```

2. Press Enter three times to accept the default file location and an empty passphrase (an empty passphrase is fine for a personal computer; add one for extra safety if you like).

3. Copy the PUBLIC key to your clipboard:

```
# Windows (Git Bash):  
clip < ~/.ssh/id_ed25519.pub  
  
# Mac:  
pbcopy < ~/.ssh/id_ed25519.pub
```

- 4.** On GitHub: click your profile photo (top right) -> Settings -> SSH and GPG keys -> New SSH key.
- 5.** Give it a title (e.g. My laptop), paste the key into the Key box, and click Add SSH key.
- 6.** Test the connection:

```
ssh -T git@github.com
```

Type yes if asked about a fingerprint. Success looks like: Hi your-username! You've successfully authenticated, but GitHub does not provide shell access. That message means everything works.

Important: Never share the file without .pub at the end (id_ed25519). That is your private key - the .pub file is the only one you paste into GitHub.

Option C (beginner-friendly): GitHub CLI (gh)

GitHub CLI is GitHub's official command-line tool. Its gh auth login command signs you in through the browser AND automatically sets up Git's credentials for you, so it is the quickest all-in-one option.

1. Install it:

```
# Windows (PowerShell or Command Prompt):  
winget install --id GitHub.cli  
  
# Mac:  
brew install gh
```

On Windows you can also download the installer from <https://cli.github.com> instead of using winget.

2. Check the installation:

```
gh --version
```

3. Sign in and let it configure Git for you:

```
gh auth login
```

Answer the prompts as they appear:

- Where do you use GitHub? -> GitHub.com
- Preferred protocol -> HTTPS (or SSH if you prefer).

- Authenticate Git with your GitHub credentials? -> Yes - this is the bit that makes plain git commands work.
- How would you like to sign in? -> Login with a web browser.
- Copy the one-time code it shows, paste it into the browser page that opens, and click Authorize.

4. Check that it worked:

```
gh auth status
```

You should see a Logged in to github.com line with your username. From now on, normal git clone and git push over HTTPS just work - no extra credential setup needed.

Note: Bonus: gh does much more than logging in - for example gh repo create, gh pr create and gh repo clone. Handy later, once you are comfortable with the basics.

Step 6 - Your first connection: clone or upload

6a. Clone an existing repository (download a project)

Cloning downloads a full copy of a GitHub project, including its whole history. On the GitHub page of the project, click the green Code button and copy the URL (choose HTTPS if you set up Option A or C, SSH if you set up Option B). Then run:

```
cd ~/Desktop
git clone https://github.com/username/project-name.git
```

A new folder appears with the project inside it. Done!

6b. Upload a brand-new project from your computer

1. On GitHub, click the + (top right) -> New repository. Give it a name, leave everything else as-is, and click Create repository.
2. In Git Bash / Terminal, go into your project folder and connect it to the empty GitHub repository you just made:

```
cd path/to/your-project
git init
git add .
git commit -m "My first commit"
git remote add origin https://github.com/username/project-name.git
git push -u origin main
```

- git init turns the folder into a Git repository.
- git add . stages (prepares) all files to be saved.
- git commit saves a snapshot with a message.
- git remote add origin links the folder to your GitHub repository.

- `git push -u origin main` uploads your commits to GitHub (the `-u` remembers the link, so later you just type `git push`).

Note: If `git push` says the branch is called `master` instead of `main`, run `git branch -M main` once, then push again.

You are all set - quick checklist

- GitHub account created and email verified.
- `git --version` shows a version number.
- `user.name` and `user.email` configured.
- Signed in via Git Credential Manager, GitHub CLI (`gh auth status`), or the SSH test says successfully authenticated.
- Cloned or pushed your first repository.

Next step: open the companion guide, [Git Commands Visual Guide](#), to learn the everyday commands with pictures - `add`, `commit`, `push`, `pull`, branches and merging.